Checking delivery routes against traffic delays: statistical screening versus travel-time padding in vehicle routing

Abstract

Background and objective. Delivery companies promise customers time windows but plan routes on average travel times, and one delay can make a driver miss several windows. The usual protection is to plan with slower speeds, padding every trip. This study asks whether a statistical check on an ordinary planner's routes is better protection than padding. **Methods.** A standard solver produces candidate plans with extra internal slack, and each candidate is simulated on 1000 days with time-of-day congestion and correlated random delays. A plan is accepted only if a one-sided 95% Wilson lower bound on the chance that at least 95% of customers are served on time reaches 0.95, and the accepted plan is tested on 5000 new days. A cap on the slack keeps every customer reachable. The study used 53 synthetic instances with 20 to 200 customers, 40 real Amazon delivery routes, a padding sweep from 0 to 50%, and three confirmatory studies with hypotheses published in advance. **Results.** Screening was 8.3–21.4 percentage points more reliable than 20% padding, although not significantly at 200 customers. No padding level matched it over all instances: the best, 30%, pooled 66.7% against 86.6%, and heavier padding often found no plan. On the Amazon routes, screening padded plans raised service from 59.0% to 75.3%. Seven of eight prespecified primary hypotheses were confirmed. **Conclusions.** A simple statistical screen can make an existing planner's routes more reliable without changing the planner. The delay model is still assumed and needs testing on real traffic data.

Keywords: vehicle routing; time windows; travel-time uncertainty; simulation; Wilson interval; operations research

1 Introduction

1.1 Background and context

A courier company that promises to deliver between 10:00 and 11:00 has to plan which driver visits which customers, and in what order, so that every visit falls inside its window. This is the vehicle routing problem with time windows (VRPTW), one of the most studied problems in operations research[1,2]. Planning software usually solves it with one fixed table of travel times. Real travel times are not fixed. Roads are slower in the morning rush, and random events such as an accident or a long wait at a door add delay. Because a driver visits customers one after another, a delay early in the day pushes every later visit towards the end of its window, and a plan that looked comfortable can fail.

1.2 Problem statement and rationale

The simplest protection, and a common one in practice, is to plan with a slower speed than expected. If every trip is assumed to take 20% longer, the plan has room to absorb delays. The drawback is that padding protects every trip equally, quiet off-peak trips as much as rush-hour ones, so the plan uses extra distance and sometimes extra vehicles whether or not a route is at risk. Research methods that model uncertainty directly, through chance constraints or robust optimization[3–5], spend that protection more carefully, but they need a solver built for the purpose, which most companies do not have.

1.3 Significance and purpose

This study tests a middle path. The company keeps its ordinary solver, but asks it for several candidate plans, each planned against internal deadlines brought forward by a few minutes. Customers never see these internal deadlines, so the promised windows do not change. Each candidate is then simulated on many random days, and a statistical test decides whether it is reliable enough. The best plan that passes is used. Its reliability is reported from a second, independent set of simulated days, so the number reported is not the one used to choose the plan.

1.4 Objectives

The study makes three contributions. First, it describes this screening procedure and proves that a simple cap on the internal slack prevents a failure in which a customer becomes impossible to reach in time. Second, it compares the procedure with padding at levels from 0 to 50%, measuring both reliability and cost, and shows that no single padding level does as well across instances. Third, it repeats the main comparisons in three confirmatory studies whose hypotheses were fixed and published before the experiments were run, using two different routing solvers.

1.5 Scope and limitations

Most instances are synthetic. A further test uses 40 real Amazon delivery routes, with real stops, travel times and service times but assumed two-hour delivery windows. In every test the travel-time delays follow an assumed model rather than measured traffic data, so the study shows whether the procedure works under controlled conditions, not how a particular fleet would perform.

1.6 Theoretical framework and related work

Routing with random travel times has been studied for more than three decades[6–8]. One line of work writes the requirement that customers are served on time as a chance constraint and solves the resulting stochastic program directly[9,10]; when the probability cannot be computed exactly, it can be estimated from sampled scenarios[11]. Other studies add time-dependent speeds[12,13], measure the risk of a route with an index instead of a probability[14,15], or predict travel times from context such as the time of day[16]. The internal deadlines used here are closest to self-imposed time windows, in which the carrier sets its own windows and places them so that the slack between them absorbs

delays[17,18]. These methods need a dedicated model or solver. The present study keeps the solver unchanged and adds only a sampling check with a known statistical guarantee[19,20].

2 Methods

2.1 The routing problem

A depot and n customers are given. Customer i has a demand, a service time of 10 minutes and a window $[l_i, u_i]$ in which service must start. Each vehicle carries at most 1000 units and works at most 600 minutes. A plan assigns customers to vehicles, orders the visits and fixes a departure time for each vehicle. Plans are compared on four quantities: total distance, total driver time, number of vehicles, and slack deficit, which measures how close planned arrivals come to the end of their windows. The planner minimizes a weighted sum of these four quantities, each scaled to a comparable range, with the weights $(0.5693, 0.2643, 0.1055, 0.0609)$ fixed before any experiment. Two solvers were used: OR-Tools from Google[21], a widely used routing library, and the hybrid genetic search (HGS) of the PyVRP package[22,23], which finds shorter routes. The full mathematical formulation is given in the project archive[24].

2.2 Travel times and random delays

Each day runs from 06:00 with a free-flow speed of 40 km/h. Between 07:00 and 09:00 speeds drop by a factor of 1.2, and between 06:00 and 07:00 they are slightly higher. Travel times are computed by integrating the speed over the time of the trip, so a vehicle that leaves later never arrives earlier, a property known as first-in-first-out[25,26]. On a simulated day, the time of each trip is multiplied by a random factor $\exp(\varepsilon)$ with

$$\varepsilon_{ij} = \sigma\big(\sqrt{\rho}\,\xi_0 + \sqrt{1-\rho}\,\xi_{ij}\big), \tag{1}$$

where ξ_0 is a standard normal variable shared by every trip on that day (a bad-traffic day slows everyone) and ξ_{ij} is an independent standard normal variable for the trip from i to j. The main setting uses $\sigma = 0.2$ and $\rho = 0.5$. The factor has median one, so on a typical day trips take about their planned time. A fixed set of simulated days is called a *bank*; every plan for an instance is tested on the same banks, so comparisons are paired.

2.3 What counts as reliable

A day counts as a success for a plan if at least 95% of its customers are served on time. The target is that a plan succeeds on at least 95% of days. Asking for *every* customer to be on time would be unrealistic for large instances: if each of 200 customers were independently on time with probability 0.99, all 200 would be on time on only about 13% of days, because $0.99^{200} \approx 0.134$.

A plan's success rate on 1000 simulated days is only an estimate, so the test uses a lower confidence bound. If a plan succeeds on a share $\hat{p}$ of N days, the one-sided Wilson lower bound[19]

at 95% confidence is

$$L(\hat{p}) = \frac{\hat{p} + \dfrac{z^2}{2N} - z\sqrt{\dfrac{\hat{p}(1-\hat{p})}{N} + \dfrac{z^2}{4N^2}}}{1 + \dfrac{z^2}{N}}, \qquad z = 1.645. \tag{2}$$

The Wilson bound was chosen because the simpler normal-approximation interval is unreliable when the true probability is close to one[20], which is exactly the region of interest here. With $N = 1000$ and $\hat{p} = 0.966$, for example, $L \approx 0.955$, so the plan passes; with $\hat{p} = 0.955$, $L \approx 0.943$ and it fails.

2.4 The screening procedure

The procedure has four steps.

1. For each buffer $\beta \in \{0, 5, 10, \ldots, 60\}$ minutes, ask the solver for a plan in which every customer's internal deadline is $u_i - \beta$ (never earlier than l_i). This gives up to 13 candidate plans.

2. Simulate every candidate on a screening bank of 1000 days and compute its lower bound (2).

3. If at least one candidate has $L \geq 0.95$, select the passing candidate with the best weighted score. Otherwise return a fallback plan fixed in advance: the plan with no buffer for the OR-Tools procedures, and the 20% padded plan for the HGS procedures.

4. Simulate the selected plan once on a separate validation bank of 5000 days and report its success rate.

Every simulated arrival is always judged against the window u_i promised to the customer, never against the internal deadline. Because the validation bank is not used to choose the plan, the reported reliability is an honest out-of-sample estimate. The average validated success rate over all instances, including those where the fallback was used, is called *pooled service*.

The main (capped) procedure plans its candidates at the normal speed with the planner's weighted objective. A second version, *screened conservative*, combines the screen with padding: it plans the candidates at the 20% padded speed for the shortest distance, keeps the passing candidate with the shortest distance, and falls back to the 20% padded plan when no candidate passes.

2.5 Keeping every customer reachable

Bringing deadlines forward can backfire. If customer i is far from the depot, an internal deadline $u_i - \beta$ can be earlier than the time it takes to drive there directly, and then no plan can exist at that buffer. Let $e_i = \max\{l_i, \bar{t}_{0i}\}$ be the earliest time at which service at i could start after a direct trip from the depot, where $\bar{t}_{0i}$ is the planned driving time. The *capped* procedure uses the internal deadline

$$u_i^{\text{cap}}(\beta) = \min\big\{u_i,\ \max\{l_i,\ u_i - \beta,\ e_i\}\big\}. \tag{3}$$

Lemma 1. *For every customer i and every buffer $\beta \geq 0$: (a) $u_i^{\mathrm{cap}}(\beta) \leq u_i$, so a plan that meets the capped deadlines also meets the promised windows; (b) if customer i can be reached in time by a direct trip, that is if $e_i \leq u_i$, then $e_i \leq u_i^{\mathrm{cap}}(\beta)$, so the capped deadline never makes a reachable customer unreachable; and (c) if $\beta \leq u_i - e_i$, the cap has no effect and $u_i^{\mathrm{cap}}(\beta) = \max\{l_i, u_i - \beta\}$.*

Proof. (a) holds because $u_i^{\mathrm{cap}}(\beta)$ is a minimum that includes u_i. For (b), the inner maximum is at least e_i, and $u_i \geq e_i$ by assumption, so the minimum of the two is at least e_i. For (c), $\beta \leq u_i - e_i$ gives $u_i - \beta \geq e_i$, so the inner maximum equals $\max\{l_i, u_i - \beta\}$, which is at most u_i because $l_i \leq u_i$ and $\beta \geq 0$; the outer minimum therefore leaves it unchanged. □

The lemma guarantees reachability one customer at a time. It does not guarantee that a single plan can serve all customers in time, because capacity, shift length and the order of visits still matter, and it does not guarantee that the target is met. A separate shortest-path test was used to prove, where possible, that a contracted model had no feasible plan, so that solver failures could be told apart from true infeasibility.

2.6 Comparisons

The main comparison is with padding: plan for the shortest distance at a slower speed, $40/(1+p)$ km/h, so that every trip is padded by p percent. The earlier studies used $p = 20\%$; this paper adds a sweep over $p \in \{0, 10, 20, 30, 40, 50\}\%$, with the same solver budget, to compare cost at equal reliability. Padded plans are not screened; they are used as an operator would use them. The *original* procedure screens contracted candidates without the cap, and the *capped* procedure uses (3). A third comparison, described in the Results, used a budgeted robust optimization model[4,27], a method that plans against a set of worst-case travel times.

2.7 Instances and statistics

Instances place a depot at the centre of a 100 km by 100 km square and customers uniformly at random, with demands from 10 to 49 units and windows from 90 to 149 minutes wide. The development study used 53 instances: 20 with 20 customers, 15 with 50, 10 with 100 and 8 with 200. Each solver call had 5 seconds for OR-Tools or 10,000 iterations for HGS. Because all configurations are tested on the same simulated days, differences are computed instance by instance, and intervals across instances come from a paired bootstrap with 20,000 resamples. A configuration that finds no plan scores zero on that instance, so failures count against it.

As an external check, the HGS procedure was also run on standard benchmark instances: the 20 Gehring–Homberger instances of classes R1 and RC1 with 200 customers[28], which extend Solomon's classic test set[1]. Their coordinates and times were rescaled so that the benchmark's travel matches 40 km/h and its horizon fits a 600-minute shift starting at 06:00, and the same delay model was added. This check was exploratory.

The second external check uses real delivery routes from the public data set of the 2021 Amazon Last Mile Routing Research Challenge[29]. The 40 routes that Amazon rated as high quality were drawn with a fixed seed from 17 depots in five US cities; each is one van's day, with 80 to 194 stops (median 150). The data give the travel time between every pair of stops, the planned service time and volume of every package, the van's capacity, its departure time and the order the driver

actually drove. Consecutive stops in the same Amazon delivery zone were joined into one customer, giving 10 to 36 customers per route; the solver rounds every trip up to a whole minute, and on single stops a few hundred metres apart that rounding alone would add more than an hour per route. A customer's service time (its packages plus the walking and driving between its stops) was added to the trips leaving it, so it receives the same random delays as driving. In Amazon's data only 7% of these stops have a delivery window, and the middle half of those windows are 7 to 8 hours wide, so each customer was offered a two-hour window containing the time the driver actually arrived, placed at random around it. The shift starts at the real local departure time, lasts 1.3 times the actual route, and up to three vans of the real capacity may be used. The unchanged OR-Tools pipeline was run on each route. This check was also exploratory.

2.8 Confirmatory studies

The development study was exploratory: its results were used to design the method. To check that the findings were not a product of that tuning, three confirmatory studies were run on new instances. Before each one, its hypotheses, tests, decision rules and instance seeds were written down and published with a cryptographic fingerprint of the code[30–32], so the analysis could not be adjusted after seeing the results. Study 1 tested the OR-Tools procedures on 53 new instances; study 2 used 53 more instances and the HGS solver; and study 3 compared the procedures with the robust model on the 90 earlier confirmatory instances with up to 100 customers (part A), and tested a new fallback rule on 53 further instances (part B). Each study, and each part of study 3, had two primary hypotheses, judged with 97.5% intervals so that the chance of a false confirmation stays below 5% for each pair.

3 Results

3.1 Screening beats 20% padding

Table 1 compares the procedures with 20% padding on the 46 development instances where padding found a plan. The original screening procedure was 8.3–21.4 percentage points more reliable, depending on size. The interval at 200 customers includes zero, so the gain is clear only up to 100 customers. Using the weighted objective without any screening was *worse* than padding at every size, which shows that the gain comes from testing candidates, not from the objective alone. The advantage grew with the amount of random delay: for independent delays ($\rho = 0$), the difference rose from -3.1 points at $\sigma = 0.1$, where padding was slightly ahead, to 30.5 points at $\sigma = 0.3$ (Fig. 1).

3.2 Padding at other levels

A fixed 20% pad may simply be the wrong amount, so Table 2 and Fig. 2 repeat the comparison with padding from 0 to 50%. Over all instances, no padding level came close to the capped screening procedure. The best level, 30%, pooled 66.7% against 86.6%, a difference of -19.9 points $[-28.8, -11.7]$. More padding did not keep helping, because heavy padding makes many

Table 1. Paired differences in service probability (percentage points) on the 46 development instances where 20% padding found a plan. SC denotes screened conservative speed with its prespecified fallback.

n	S minus C	P minus C	SC minus C
20	−11.0 [−16.1, −6.0]	21.4 [9.6, 31.3]	28.2 [21.4, 34.3]
50	−13.1 [−17.4, −8.9]	17.9 [2.8, 32.0]	11.8 [2.7, 22.2]
100	−16.2 [−20.6, −11.7]	16.0 [4.8, 24.8]	11.2 [2.8, 19.4]
200	−12.9 [−18.5, −5.7]	8.3 [−5.1, 20.0]	5.7 [0.0, 12.9]

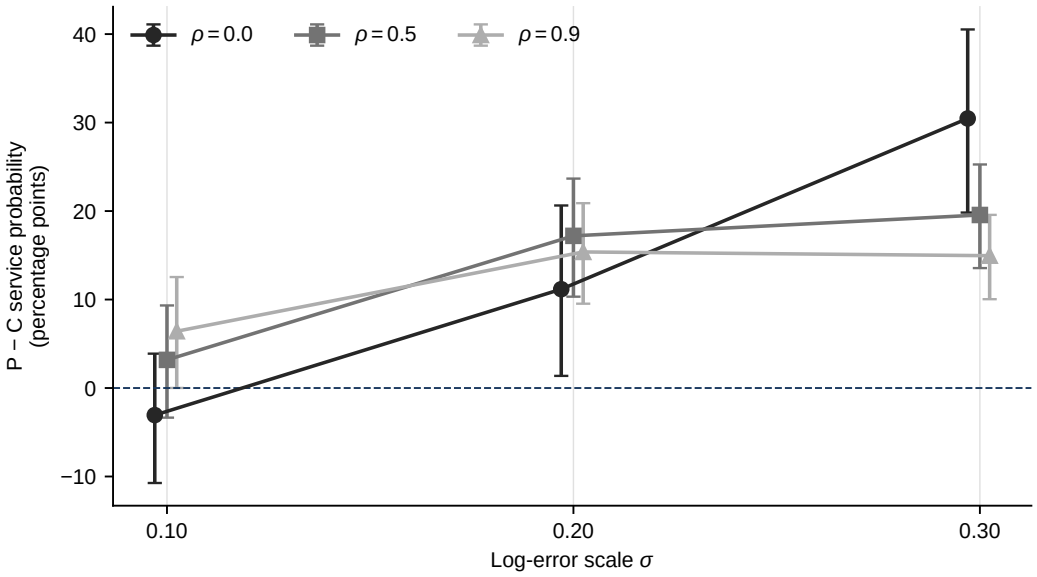


Figure 1. Difference in validated service probability between the original screening procedure (P) and 20% padding (C) on the 46 development instances, as the random delay σ and its correlation ρ vary. Plans are fixed; error bars are 95% bootstrap intervals over instances.

instances impossible to plan: the padded planner found a plan on 34 of the 53 instances at 40% and on only 26 at 50%.

Where heavy padding did find a plan, that plan was reliable. On those 26 instances, 50% padding served 96.1% against 87.9% for screening, 8.2 points more [1.2, 16.1], and used 0.77 more vehicles per instance. These were the easier instances, however, and on the other 27 the padded planner returned nothing. A planner who fixes one padding level therefore has to choose between unreliable plans at low levels and missing plans at high ones. Screening avoids that choice, because it picks the buffer for each instance separately and the cap keeps every customer reachable. At 30% padding, where the padded planner did best overall, screening drove 3.1% more distance than the padded plans on the instances where both had a plan, so its extra reliability costs only a few percent of distance.

Table 2. Padding every trip by a fixed percentage on the 53 development instances (exploratory). *Plans*: instances on which the padded planner found a plan. *All*: pooled service over all instances, an instance without a plan scoring zero. The next three columns use only the instances with a padded plan: the padded plans' service, the capped screening procedure's service on the same instances, and the difference (padding minus screening, percentage points, 95% paired bootstrap interval). Distance (%) and vehicles are also padding minus screening on those instances. Over all instances the capped procedure pooled 86.6%.

		All	Instances with a padded plan				
Padding	Plans	Pooled	Padded	Screened	Difference	Distance	Vehicles
0%	53/53	21.9	21.9	86.6	−64.7 [−70.2, −58.8]	−7.1	−0.89
10%	51/53	43.7	45.4	88.4	−42.9 [−48.0, −37.4]	−6.2	−0.59
20%	46/53	56.8	65.4	89.4	−23.9 [−29.2, −18.2]	−4.8	−0.22
30%	43/53	66.7	82.2	90.3	−8.1 [−13.1, −2.4]	−3.1	0.21
40%	34/53	58.8	91.6	88.5	3.1 [−2.8, 10.0]	−1.0	0.47
50%	26/53	47.1	96.1	87.9	8.2 [1.2, 16.1]	0.9	0.77

Table 3. Development study, all 53 instances. For each procedure: instances admitted by screening out of all instances of that size, with the number whose lower bound also reached 0.95 on the validation bank in parentheses, and pooled validated service probability (%) with a 95% bootstrap interval, fallback plans included.

	Original procedure		Screened conservative		Capped procedure	
n	Admitted	Pooled	Admitted	Pooled	Admitted	Pooled
20	14/20 (14)	81.6 [70.2, 91.9]	15/20 (15)	87.7 [79.0, 94.7]	14/20 (14)	81.6 [70.3, 92.0]
50	8/15 (8)	72.7 [59.2, 86.4]	4/15 (4)	67.5 [58.2, 77.3]	10/15 (10)	80.1 [66.9, 90.8]
100	6/10 (5)	81.3 [68.8, 93.3]	4/10 (4)	77.5 [67.2, 88.2]	10/10 (9)	97.5 [96.7, 98.2]
200	4/8 (3)	80.6 [68.7, 92.3]	2/8 (2)	78.3 [71.0, 86.5]	8/8 (7)	97.6 [96.9, 98.2]

3.3 The cap removes most failures

On 21 development instances the original procedure found no plan that passed the test, and on 17 of them the shortest-path test proved that some customer had become unreachable at a large buffer. Capping (Table 3) raised the number of instances with a passing plan from 32 to 42, and pooled service by 7.7 [3.5, 12.5] points. At 100 and 200 customers the capped procedure found a passing plan on every instance and pooled 97.5–97.6%. It used 2.5% more distance and 0.43 more vehicles on average than the original procedure. Of the 42 plans that passed screening, 40 also kept a lower bound of at least 0.95 on the 5000 validation days, which shows that the screen rarely admits a plan that later disappoints.

Pooled service stayed below the 95% target because of the remaining instances, all with 20 or 50 customers, where no candidate passed and a fallback was used. Plans that passed averaged 97.4% validated service, while the 11 fallback plans averaged 45.2%.

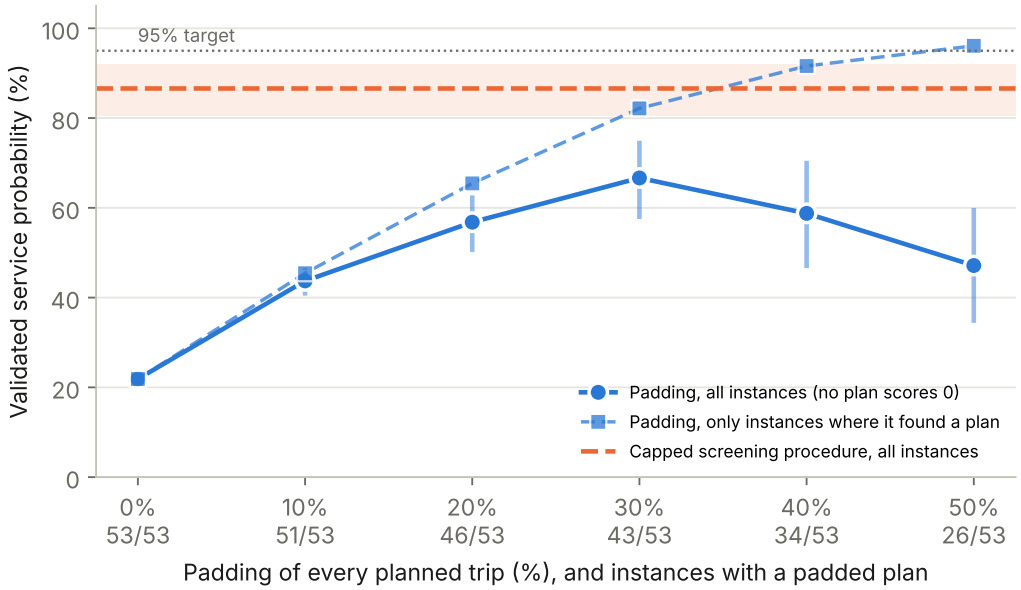


Figure 2. Validated service probability against the padding applied to every trip, on the 53 development instances. The solid line counts an instance without a padded plan as zero; the dashed line uses only the instances where a padded plan exists (counts under the axis). The orange line and band show the capped screening procedure over all instances with its 95% bootstrap interval.

Table 4. Gehring–Homberger instances with 200 customers (exploratory, HGS pipeline): instances, instances where the anchor solves found a plan, instances admitted by screening, and pooled validated service-event probability (%) of the HGS nominal plan, HGS conservative speed and the HGS capped procedure with conservative fallback. A missing plan scores zero.

Class	Inst.	Anchors	Admitted	Nominal	Conservative	Capped
R1	10	8	7	42.1	68.0	77.0
RC1	10	10	10	27.6	79.3	97.4

3.4 Standard benchmark instances

On R1_2_1 and R1_2_2, where all or most windows are only 10 time units wide, neither the solver runs that seed the procedure nor HGS at conservative speed found a plan, so the procedure could not run there. Screening admitted a plan on 17 of the other 18 instances (Table 4). On those 18 instances the capped HGS procedure exceeded conservative speed by 15.1 [11.0, 19.1] points, and over all 20 it pooled 87.2% against 73.6% for conservative speed. The gain on these harder, clustered benchmark instances is close to the one on the synthetic instances.

3.5 Real delivery routes

On the Amazon routes, both screened procedures were more reliable than 20% padding (Table 5). Screened conservative pooled 75.3% against 59.0% for padding, a difference of 16.3 [11.0, 22.2] points, and the capped procedure pooled 72.6%, 13.6 [3.9, 24.4] points above padding. Part of the

Table 5. Real delivery routes: 40 Amazon routes from five US cities (exploratory). *Plans*: routes on which the configuration produced a plan; *Admitted*: routes on which a candidate passed the screen. Pooled validated service probability (%) with a 95% bootstrap interval over routes; a route without a plan scores zero.

Configuration	Plans	Admitted	Pooled service
Nominal plan (no protection)	34/40	–	41.4 [35.1, 47.1]
20% padding	28/40	–	59.0 [46.6, 70.8]
Capped screening procedure	34/40	24	72.6 [61.2, 82.9]
Screened conservative	34/40	25	75.3 [63.6, 85.7]

gap comes from planning: padding found no plan on 12 routes, against 6 for the other configurations. Where the capped procedure admitted a plan (24 routes), its validated service averaged 97.4%, and on 23 of them it was at least 95%. On the 10 routes where no candidate passed, its fallback averaged 56.8% against 62.9% for padding, and on the 28 routes where both had a plan the two were not distinguishable (1.6 $[-4.4, 7.2]$ points). Screened conservative avoids that weakness because its fallback is the padded plan. On 6 routes with 22 to 36 customers no configuration found a plan within the time limit, even though the driver's own order is feasible by construction, so these failures are limits of the solver's search, not of the instances.

3.6 Confirmatory studies

Table 6 lists the eight primary hypotheses. Seven were confirmed. In study 1, screening beat 20% padding by 15.7 points on new instances, and capping added 11.7 points. In study 2, with the stronger HGS solver, capped screening beat padding by 28.0 points, so the gain does not depend on a weak solver. The hypothesis that was not confirmed (G2) concerned a change to the fallback: returning the padded plan when nothing passes did not measurably help the capped OR-Tools procedure, because that procedure rarely needed a fallback. Study 3 part B tested a different rule, returning the candidate that came closest to passing, and it raised pooled service by 7.3 and 4.2 points for the two solvers, to 94.4% and 94.9%.

3.7 Comparison with robust optimization

A budgeted robust model plans against worst-case travel times on a limited number of trips. Without a reachability correction it failed on 16 of 45 development instances at its most protective setting, for the same reason the uncapped procedure failed. With the analogous correction and the same screen, the robust plans were more reliable than both screened procedures in study 3 part A (by 6.7 and 6.7 points; Table 6), but they used 0.42 more vehicles per instance than the HGS procedure. When they were limited to no more vehicles than the comparison plan, their advantage was no longer statistically detectable. Screening is therefore not the most reliable planner tested; its strength is that it works with any existing solver.

Table 6. The eight primary hypotheses of the three confirmatory studies. Each was fixed and publicly timestamped before its instances were solved. Estimates are differences in validated service probability (percentage points) with 97.5% paired bootstrap intervals over instances; a hypothesis is confirmed when its interval lies above zero.

Study		Contrast	Estimate [interval]	Decision
1	H1	Original procedure minus 20% padding (50 instances)	15.7 [7.5, 23.6]	confirmed
1	H2	Capped minus original procedure (53 instances)	11.7 [5.9, 17.9]	confirmed
2	G1	Capped procedure with HGS minus 20% padding with HGS (49 instances)	28.0 [23.0, 32.8]	confirmed
2	G2	Conservative minus slack-aware fallback, capped OR-Tools procedure	0.4 [−0.7, 2.1]	not confirmed
3A	R1	Screened robust model minus capped OR-Tools procedure (90 instances)	6.7 [3.0, 11.1]	confirmed
3A	R2	Screened robust model minus capped HGS procedure (45 instances)	6.7 [2.2, 12.2]	confirmed
3B	F1	Best-screened minus default fallback, capped OR-Tools procedure	7.3 [2.3, 13.2]	confirmed
3B	F2	Best-screened minus default fallback, capped HGS procedure	4.2 [1.2, 8.0]	confirmed

4 Discussion

4.1 Key findings and implications

The results support a practical point. A company that already uses a routing solver can make its plans more reliable than a fixed 20% padding without changing the solver, by asking it for a few buffered variants and keeping the best one that passes a simulation test. The test is cheap: screening a candidate took 3 to 24 ms, against several seconds to generate it. Each accepted plan also comes with a stated, tested reliability, which padding does not provide. The padding sweep shows why choosing the buffer per instance matters: a fixed pad is either too small to protect most routes or so large that many instances can no longer be planned. The reachability cap is a one-line change, and Lemma 1 shows why it is safe; the same idea also rescued the robust model.

4.2 Limitations

The study has clear limits. The cities are synthetic squares and the delay model is assumed, not estimated from real traffic data, so the percentages are not forecasts for a real fleet. The benchmark and Amazon checks show that the result is not specific to the instance generator, but even the Amazon routes use assumed delivery windows and an assumed delay model. The confirmatory studies show that the findings repeat on new instances from the same generator; they do not show that they hold in a different city. The advantage also depends on the planner's objective: with equal weights on the four quantities it fell to 6.3 points, with an interval that included zero. The Wilson

bound holds for each candidate separately, not for all 13 at once; a stricter Bonferroni version removed all 11 admitted plans that later fell short, while 1 new selection fell short, at a cost of at most 2 admissions per study. Finally, vehicles always leave at their planned times, and breakdowns and same-day replanning are not modelled.

4.3 Recommendations

Pooled service stayed just below 95% because of the few instances with no passing plan. Better candidates for those instances, for example from a solver that models uncertainty directly, are the clearest next step, together with a test on real travel-time data. The Amazon routes also show that the fallback matters: when no candidate passes, returning the padded plan is safer than returning the unbuffered one, so a company adopting the screen should use the padded plan as its fallback.

4.4 Closing thought

On simulated delivery problems with 20 to 200 customers, screening a family of buffered plans from an ordinary solver gave more reliable service than padding every trip by 20%, with two different solvers; pooled over all instances it also beat every fixed padding level from 0 to 50%, and three confirmatory studies with published hypotheses confirmed seven of their eight primary hypotheses. A simple cap on the buffer, proved safe in Lemma 1, removed a failure that also affects robust optimization. The method's main appeal is that it adds a measurable reliability guarantee to the tools companies already use.

AI Usage Disclosure

Two generative AI assistants were used: Anthropic's Claude and OpenAI's ChatGPT. They were used to write and debug the simulation and analysis code, to review that code, to run and check the statistical analyses, to search the literature, and to draft and revise the text of this manuscript, including its wording. The research question, the decisions on study design and the confirmatory protocols were the author's. Every experiment was run from the archived code, every reported number is regenerated from the archived records, and the author takes full responsibility for the content.

References

[1] M. M. Solomon. Algorithms for the vehicle routing and scheduling problems with time window constraints. *Oper Res*, 35(2):254–265, 1987. doi: 10.1287/opre.35.2.254.

[2] P. Toth and D. Vigo, editors. *Vehicle Routing: Problems, Methods, and Applications*. MOS-SIAM Series on Optimization. SIAM, Philadelphia, 2nd edition, 2014. doi: 10.1137/1.9781611973594.

[3] F. Errico, G. Desaulniers, M. Gendreau, W. Rei, and L.-M. Rousseau. The vehicle routing problem with hard time windows and stochastic service times. *EURO J Transp Logist*, 7(3): 223–251, 2018. doi: 10.1007/s13676-016-0101-4.

[4] A. Agra, M. Christiansen, R. Figueiredo, L. M. Hvattum, M. Poss, and C. Requejo. The robust vehicle routing problem with time windows. *Comput Oper Res*, 40(3):856–866, 2013. doi: 10.1016/j.cor.2012.10.002.

[5] M. Gendreau, O. Jabali, and W. Rei. 50th anniversary invited article—Future research directions in stochastic vehicle routing. *Transp Sci*, 50(4):1163–1173, 2016. doi: 10.1287/trsc.2016.0709.

[6] G. Laporte, F. Louveaux, and H. Mercure. The vehicle routing problem with stochastic travel times. *Transp Sci*, 26(3):161–170, 1992. doi: 10.1287/trsc.26.3.161.

[7] M. Gendreau, G. Laporte, and R. Séguin. Stochastic vehicle routing. *Eur J Oper Res*, 88(1): 3–12, 1996. doi: 10.1016/0377-2217(95)00050-X.

[8] A. S. Kenyon and D. P. Morton. Stochastic vehicle routing with random travel times. *Transp Sci*, 37(1):69–82, 2003. doi: 10.1287/trsc.37.1.69.12820.

[9] X. Li, P. Tian, and S. C. H. Leung. Vehicle routing problems with time windows and stochastic travel and service times: Models and algorithm. *Int J Prod Econ*, 125(1):137–145, 2010. doi: 10.1016/j.ijpe.2010.01.013.

[10] J. F. Ehmke, A. M. Campbell, and T. L. Urban. Ensuring service levels in routing problems with time windows and stochastic travel times. *Eur J Oper Res*, 240(2):539–550, 2015. doi: 10.1016/j.ejor.2014.06.045.

[11] S. Ahmed and A. Shapiro. Solving chance-constrained stochastic programs via sampling and integer programming. In Z.-L. Chen and S. Raghavan, editors, *State-of-the-Art Decision-Making Tools in the Information-Intensive Age*, TutORials in Operations Research, pages 261–269. INFORMS, Hanover, MD, 2008. doi: 10.1287/educ.1080.0048.

[12] D. Taş, N. Dellaert, T. van Woensel, and A. G. de Kok. The time-dependent vehicle routing problem with soft time windows and stochastic travel times. *Transp Res Part C Emerg Technol*, 48:66–83, 2014. doi: 10.1016/j.trc.2014.08.007.

[13] F. Bomboi, C. Buchheim, and J. Pruente. On the stochastic vehicle routing problem with time windows, correlated travel times, and time dependency. *4OR*, 20(2):217–239, 2022. doi: 10.1007/s10288-021-00476-z.

[14] P. Jaillet, J. Qi, and M. Sim. Routing optimization under uncertainty. *Oper Res*, 64(1):186–200, 2016. doi: 10.1287/opre.2015.1462.

[15] Y. Zhang, R. Baldacci, M. Sim, and J. Tang. Routing optimization with time windows under uncertainty. *Math Program*, 175(1–2):263–305, 2019. doi: 10.1007/s10107-018-1243-y.

[16] B. Serrano, A. M. Florio, S. Minner, M. Schiffer, and T. Vidal. Contextual stochastic vehicle routing with time windows. *INFORMS J Comput*, 2026. doi: 10.1287/ijoc.2025.1189. Articles in advance.

[17] O. Jabali, R. Leus, T. Van Woensel, and T. de Kok. Self-imposed time windows in vehicle routing problems. *OR Spectrum*, 37(2):331–352, 2015. doi: 10.1007/s00291-013-0348-1.

[18] A. D. Vareias, P. P. Repoussis, and C. D. Tarantilis. Assessing customer service reliability in route planning with self-imposed time windows and stochastic travel times. *Transp Sci*, 53(1): 256–281, 2019. doi: 10.1287/trsc.2017.0748.

[19] E. B. Wilson. Probable inference, the law of succession, and statistical inference. *J Am Stat Assoc*, 22(158):209–212, 1927. doi: 10.1080/01621459.1927.10502953.

[20] L. D. Brown, T. T. Cai, and A. DasGupta. Interval estimation for a binomial proportion. *Stat Sci*, 16(2):101–133, 2001. doi: 10.1214/ss/1009213286.

[21] L. Perron and V. Furnon. OR-Tools, 2026. URL `https://developers.google.com/optimization`. Version 9.15.

[22] T. Vidal. Hybrid genetic search for the CVRP: Open-source implementation and SWAP* neighborhood. *Comput Oper Res*, 140:105643, 2022. doi: 10.1016/j.cor.2021.105643.

[23] N. A. Wouda, L. Lan, and W. Kool. PyVRP: A high-performance VRP solver package. *INFORMS J Comput*, 36(4):943–955, 2024. doi: 10.1287/ijoc.2023.0055.

[24] Anonymous. Public code and data archive, 2026. Details removed for review.

[25] S. Ichoua, M. Gendreau, and J.-Y. Potvin. Vehicle dispatching with time-dependent travel times. *Eur J Oper Res*, 144(2):379–396, 2003. doi: 10.1016/S0377-2217(02)00147-9.

[26] G. Ghiani and E. Guerriero. A note on the Ichoua, Gendreau, and Potvin (2003) travel time model. *Transp Sci*, 48(3):458–462, 2014. doi: 10.1287/trsc.2013.0491.

[27] D. Bertsimas and M. Sim. The price of robustness. *Oper Res*, 52(1):35–53, 2004. doi: 10.1287/opre.1030.0065.

[28] H. Gehring and J. Homberger. A parallel hybrid evolutionary metaheuristic for the vehicle routing problem with time windows. In K. Miettinen, M. Mäkelä, and J. Toivanen, editors, *Proceedings of EUROGEN99*, pages 57–64, Jyväskylä, 1999. University of Jyväskylä.

[29] Daniel Merchán, Jatin Arora, Julian Pachon, Karthik Konduri, Matthias Winkenbach, Steven Parks, and Joseph Noszek. 2021 Amazon last mile routing research challenge: data set. *Transp Sci*, 2022. doi: 10.1287/trsc.2022.1173.

[30] Anonymous. Public protocol record, 2026. Details removed for review.

[31] Anonymous. Public protocol record, 2026. Details removed for review.

[32] Anonymous. Public protocol record, 2026. Details removed for review.